Go2 の強化学習歩行と疑似 LiDAR による巡回・障害物回避

― Genesis 上での 3 層構成(歩行・反応的回避・占有格子 + A*)と 45 配置による定量評価 ―

Physical AI 2026 応用編1 最終課題 説明資料 ／ Omnicampus アカウント: **Kaoru_Kyoto** ／ コード: `https://github.com/deepkick/go2-genesis-patrol` ／ 動画: `PAI最終課題_Kaoru_Kyoto`(YouTube、約 2 分、一部 2 倍速)

1. 概要

四足歩行ロボット Unitree Go2 を物理シミュレータ Genesis 上で扱い、(1) 速度指令に追従する歩行方策を強化学習(PPO、4096 環境並列)で獲得し、(2) その上に、胴体に取り付けた疑似 2D LiDAR の距離だけを用いる障害物回避と経路追従のナビゲーション層を載せて、2 m 角の正方形経路を巡回させた。ナビゲーション層は **v1 反応的回避 → v2 コミット区間 → v3 占有格子地図 + A* 大域計画**の 3 段階で系統的に改良し、各段階を障害物をランダム配置した **45 配置**で定量評価した。巡回成功率は **9%(回避なし)→ 91%(v1)→ 96%(v2)→ 100%(v3)**、障害物への物理接触率は **91% → 0%**(同一配置での対応あり McNemar 正確検定 $p < 0.0001$、転倒 0 件)。v3 は v1 の速さと v2 の確実性のトレードオフを解消し、成功分の所要時間は v1 より中央値 3.4 s、v2 より 6.1 s 短い(符号検定 $p < 0.0001$)。評価は物理接触を主指標とし、Wilson 信頼区間と対応あり検定、シミュレータの非決定性の切り分けを含めて設計した。実機 Go2 の「自己位置 → 地図・経路計画 → 速度指令 → 歩行コントローラ」と同じ層構造を再現しており、歩行(学習)とナビゲーション(決定的・説明可能)を疎結合に分けている点が特徴である。全工程は M1 Max の Mac 1 台(Metal / CPU)で完結し、GPU サーバやクラウドは使っていない。

2. 目的と背景

実機の自律巡回ロボットは「自己位置推定 → 経路計画 → 速度指令 → 歩行コントローラ」の層構造で作られ、下層ほどロボット固有、上層ほどタスク固有である。本課題では、このうち**速度指令 → 歩行**を強化学習で、**センサ → 速度指令**を古典的な制御(反応的回避、最終的には占有格子地図 + A*)で構成し、シミュレータ上で同じ構造を再現することを目的とした。狙いは次の 3 点である。

- **タスク非依存の歩行方策**: 方策は「与えられた (v_x, v_y, ω_z) に追従する」ことだけを学ぶ。巡回・回避は上層で差し替え可能にする。
- **センサの生データだけで避ける**: 障害物の座標は制御に使わず、LiDAR の距離配列のみを入力とする(障害物座標はシーン生成と採点にのみ使用)。v3 の地図も LiDAR から逐次構築する。
- **定量評価**: 1 回の成功動画ではなく、配置をランダム化した多数のシーンで成功率・接触率・所要時間・経路効率を測り、改良の効果を対応あり検定で判定する。

3. システム構成

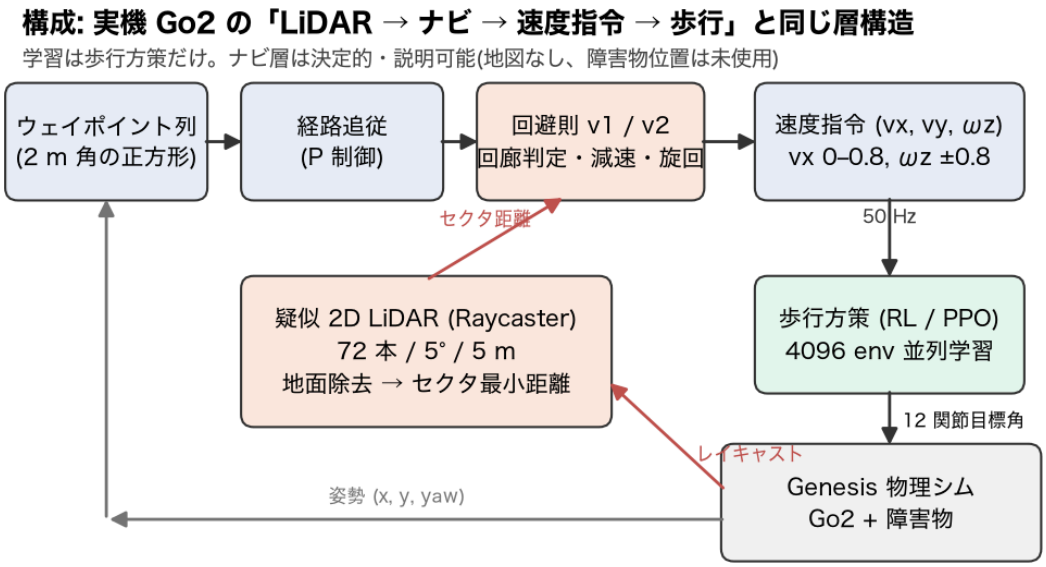


図 1: システム構成(v1/v2)。学習するのは歩行方策のみ。ナビゲーション層(経路追従・回避則)は決定的で、入力は LiDAR のセクタ距離と真の姿勢。v3 では LiDAR → 占有格子地図 → A* → ローカル目標 の経路が経路追従の前段に入り、回避則は下層の安全停止として残る(§8)。

層	実装	実機 Go2 との対応
経路追従	4 つのウェイポイント(2 m 角)への P 制御。進行方向誤差 → ω_z、距離 → v_x。到達判定 0.3 m	Nav2 のローカル追従に相当
地図 + 大域計画(v3)	地面除去後の LiDAR ヒット点から log-odds 占有格子(0.05 m、120×120)を逐次更新。占有セルを Go2 の半径 0.35 m で膨張し、8 近傍 A* で次のウェイポイントまでの経路を 1 s ごとに再計画。経路上 0.8 m 先をローカル目標として経路追従に渡す	Nav2 の costmap + global planner
障害物回避(v1 / v2、v3 の下層)	進路幅 ±0.4 m の「回廊」内に入った LiDAR ヒットのみを正面障害物とみなし、1.0 m 未満で減速、0.45 m 未満で停止・その場旋回。旋回方向は最近接ヒットの反対側。ω_z の変化率を ±0.1 rad/s/step に制限	反応的なローカル回避(DWA/VFH の簡易版)
疑似 2D LiDAR	Genesis の Raycaster を胴体リンクに取付(高さ +0.15 m)。360°/5° 刻み 72 本、最大 5 m。ヒット点の $z < 5$ cm を床とみなして除去(地面除去)。12 セクタの最小距離に集約	3D LiDAR を 2D スキャンに落としたもの。地面除去は実機の前処理と同じ
歩行方策	PPO(rsl-rl)。観測 45 次元(胴体角速度・重力方向・速度指令・関節角/角速度・前回行動)、行動 12 次元(関節目標角の補正)、50 Hz、PD 制御(k_p=20, k_d=0.5)	Go2 の歩行コントローラ
シミュレータ	Genesis 1.4.4(Apache-2.0)。学習は Metal バックエンド(4096 環境)、評価・録画は CPU バックエンド(1 環境)	—

速度指令の範囲は $v_x \in [0, 0.8]$ m/s、$v_y \in [-0.2, 0.2]$ m/s、$\omega_z \in [-0.8, 0.8]$ rad/s で、回避則の出力もこの範囲にクリップする。衝突判定は Genesis の接触情報(`get_contacts`)を用いる。

4. 歩行方策の学習

Genesis 付属の Go2 歩行例題(Legged Gym 由来の報酬: 指令速度への追従 + 上下動・姿勢・行動変化のペナルティ)を出発点にした。ただし既定の例題は速度指令が **前進 0.5 m/s に固定**されており、「前進しかできない方策」しか得られない。巡回には減速・停止・旋回が必要なので、指令範囲を上記のとおり広げ、4 秒ごとに再

サンプリングする設定で **4096 環境 × 300 イテレーション**を学習した(M1 Max、Metal、約 9 分、7.4〜7.8 万 steps/s)。

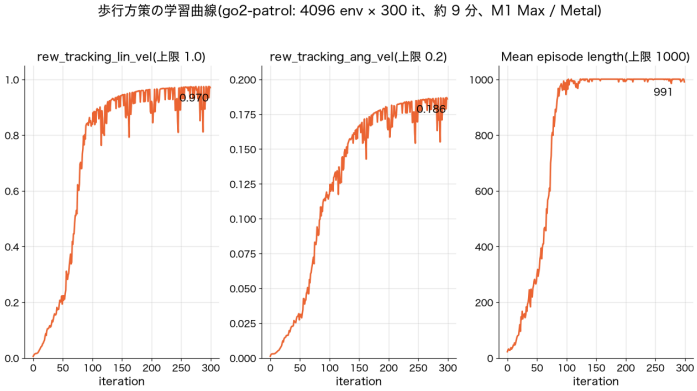


図 2: 学習曲線。速度追従報酬 0.97/1.0、旋回追従 0.186/0.2、エピソード長 991/1000(転倒ほぼゼロ)。

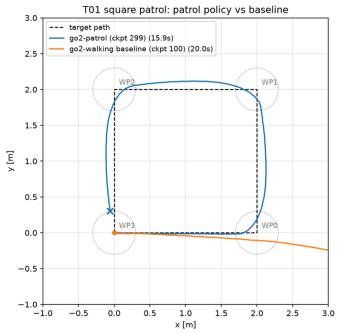


図 3: 正方形巡回(障害物なし)。拡張方策(青)は 4 隅を 15.9 s で回る。既定方策(橙)は旋回指令を無視して直進し 1/4 で時間切れ。

図 3 のとおり、既定方策は WP0 到達後に旋回指令に追従できず直進する。**巡回には指令範囲を広げた再学習が必須**であり、これは後述の 20 配置評価でも既定方策が全条件で成功 0% だったことで裏付けられた。

5. 疑似 LiDAR と反応的回避

Raycaster を既知配置(正面 1.3 m に箱、右 1.85 m に円柱)で単体検証し、距離が幾何計算の期待値と 2 mm 以内で一致すること、Metal と CPU で値が完全一致することを確認した。重要だった落とし穴が 2 つある。(1) 胴体が前傾すると水平レイが床に当たり、2〜5 m 先に偽の障害物が見える。ヒット点の高さで床を除去する処理を入れ、障害物なしの巡回 802 ステップで除去後の偽ヒットが 0 本になることを確認した。(2) 初期姿勢(落下前)はセンサが高さ 0.5 m の箱を越えて見えないため、障害物の高さは 0.6 m 以上とし、着地後からスキャンを使う。

回避則は当初「正面 ±45° と左右セクタの空き具合で旋回方向を決める」設計だったが、3 配置の試験で 2 配置が未到達となった。ログを調べると、「正面右の障害物に対して、左の遠くに別の障害物があるという理由で右(障害物側)へ旋回する」欠陥があった。そこで正面判定を **進路幅 ±0.4 m の回廊**に、旋回方向を **最近接ヒットの反対側**に変更した(v1)。これにより 3 配置すべてで 4/4 到達・接触 0 となった。

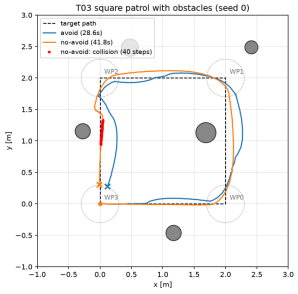


図 4: 障害物あり配置(seed 0)。回避あり(青)は経路脇の円柱を内側に避ける。回避なし(橙)は同じ円柱に 25 秒押し付けられる(赤点)。

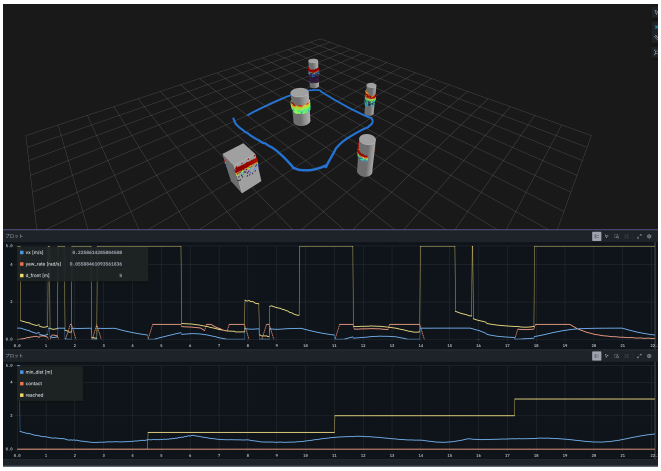


図 5: 巡回ログ(MCAP)を Foxglove で可視化。地面除去後の LiDAR 点群(z で色付け)が 5 つの障害物の側面をなぞり、青い軌跡がその間を通る。下段は v_x・ω_z・正面距離の時系列。

巡回中の全データ(TF、スキャン、点群、軌跡、障害物、指令)は Foxglove SDK で MCAP に書き出せるようにした(ROS 不要)。図 5 は実機の LiDAR 運用と同じ可視化経路で、点群が障害物の輪郭を描き、正面距離が下が

ると v_x が落ちて ω_z が立つ因果が時系列で読める。

6. 評価設計

- **シーン**: 2 m 角の正方形経路の周囲 3 m 角に、箱(0.3〜0.5 m 角)と円柱(半径 0.1〜0.2 m)を 3〜6 個乱数配置(seed で再現可能)。ウェイポイントから 0.5 m、初期位置から 1.0 m、障害物同士 1.2 m のクリアランスを棄却サンプリングで確保。
- **指標**: 成功 = 60 秒以内に 4 ウェイポイント到達 かつ 転倒なし かつ **物理接触なし**。接触率、ニアミス率(LiDAR 最小距離 < 0.25 m)、成功分の所要時間、経路効率(直線距離 8.0 m / 走行距離)。
- **衝突判定の変更**: 設計当初は LiDAR 最小距離 < 0.25 m を衝突としていたが、正面から押し付けている状態では胴体前端が当たっていてもセンサ中心から 0.3 m 以上あり検出できないことが分かった(横擦りは検出できる)。そこで物理接触を主指標、LiDAR 判定を副指標に改めた。
- **比較**: 同一 seed に対して条件を変える対応あり比較。McNemar 正確検定(成功・接触)、Wilson 95% 信頼区間、所要時間は符号検定・Wilcoxon 符号順位検定(数式を自前実装)。
- **非決定性**: 同一条件でも結果が揺れること(例: ある配置で到達数が 1/4〜4/4)を確認したため、録画の有無・MCAP 出力の有無・プロセスの別など 5 条件 × 3 回で切り分けた。条件間に系統差はなく、CPU バックエンドの真の非決定性と判断した。評価は配置数で平均化する設計のため結論には影響しないが、「1 配置の 1 回の結果」は信用しない。

7. 結果

7.1 回避則の効果と方策との交互作用(20 配置、2×2)

障害物	歩行方策	回避	n	成功率	接触率	所要(成功分)
なし	既定(前進のみ)	なし / あり	5+5	0% / 0%	0% / 0%	—
なし	拡張	なし / あり	5+5	100% / 100%	0% / 0%	15.9 / 16.1 s
あり	既定(前進のみ)	なし / あり	15+15	0% / 0%	47% / **67%**	—
あり	拡張	なし / **あり(v1)**	15+15	13% / **93%**	87% / **0%**	16.0 / 33.6 s

回避則は拡張方策と組み合わせたときだけ効く。**既定方策に回避則を付けると接触率がむしろ 47% → 67% に上がった**。既定方策は停止指令には従わず約 0.3 m/s で前進し続ける一方、旋回指令にはわずかに追従するため、「減速して横へ逃げる」つもりの指令が減速なしの緩い旋回になり、経路脇の障害物へ寄っていく。回避則は「指令どおりに減速・旋回できる歩行方策」を前提にしており、前提が崩れると安全側に働かない。期待と異なる結果だがそのまま報告する。

7.2 45 配置 × 4 条件(拡張方策、障害物あり seed 5–49)

条件	成功率 [95% CI]	接触率	所要 [s]	経路効率
回避なし	4/45 = 9% [4–21]	91%	15.9	1.07
回避 v1	41/45 = 91% [79–96]	0%	31.6	1.00
回避 v2(コミット区間)	43/45 = 96% [85–99]	0%	31.7	0.81
v3(占有格子 + A*)	**45/45 = 100%** [92–100]	**0%**	**24.2**	**1.05**

転倒 0/135。所要・経路効率は成功分の平均。対応あり McNemar 正確検定: 回避なし vs v1 — 成功 $p < 0.0001$(v1 のみ成功 37、なしのみ 0)、接触 $p < 0.0001$(41 vs 0)。v1 vs v2 — 成功 $p = 0.69$(v2 のみ成功 4、v1 のみ 2)、所要時間の差 中央値 +1.1 s(符号検定 $p = 0.51$、Wilcoxon $p = 0.76$)。v3 vs v1 — 成功 $p = 0.125$(v3 のみ成功 4、v1 のみ 0)、所要時間(両方成功 41 配置)中央値 −3.4 s、v3 が速い 34 / 遅い 3(符号検定・Wilcoxon とも $p < 0.0001$)。v3 vs v2 — 成功 $p = 0.5$(v3 のみ 2、v2 のみ 0)、所要時間(43 配置)中央値 −6.1 s、36 / 5($p < 0.0001$)。v1・v2 の失敗 6 配置はすべて v3 で成功し、v3 で新たに失敗した配置はない。

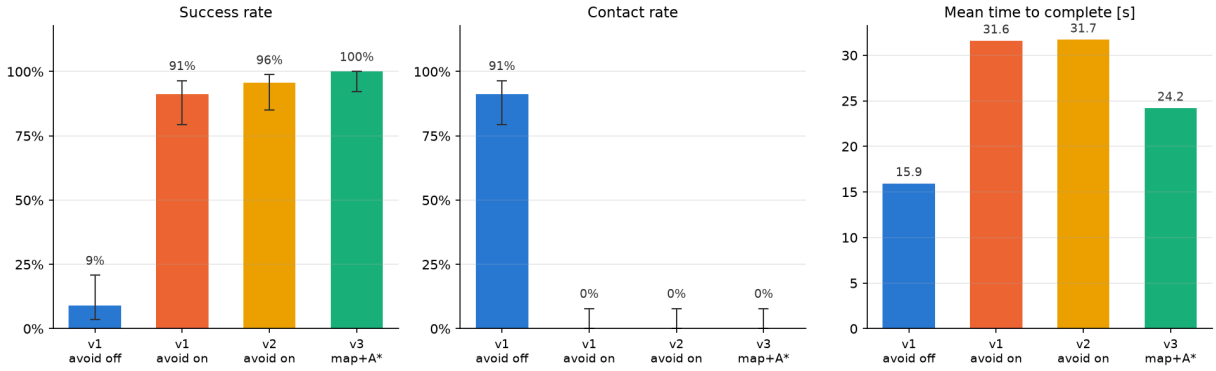


図 6: 45 配置 × 4 条件。左: 成功率、中: 接触率、右: 成功分の平均所要時間(誤差棒 = Wilson 95% CI)。

転倒 0/180。成功率の改善は回避なし → v1 で統計的に明確、v1 → v2 → v3 の差は失敗数がもともと少ないため McNemar では示せない。一方、**v3 の所要時間の短縮は対応あり比較で明確**で、v1/v2 が時間切れになった 6 配置を含む全配置を 17〜46 s で完走した。

8. 系統的改良: v1 → v2 → v3 と失敗分析

v1 の失敗はすべて同じ型だった。経路の真正面に障害物があると「停止 → その場旋回 → 回廊が空く → 経路追従が元の向きへ戻す → 再び停止」を繰り返す**振動**で、接触も転倒もないまま時間切れになる(図 7 左)。原因は、回廊が空いた瞬間に経路追従へ制御を戻すことにある。そこで v2 では、回避が発動したら旋回方向を $\mathbf{T_{commit} = 2.0\ s}$ 固定し、正面が空いても $\mathbf{T_{clear} = 1.0\ s}$ 直進してから経路追従に戻す「コミット区間」を導入した。初版は直進中に経路追従の v_x が 0 になって止まる欠陥があり、巡航速度 0.4 m/s の下限を入れて修正した(T_{commit} 1.0 s は悪化、2.0 s で改善)。

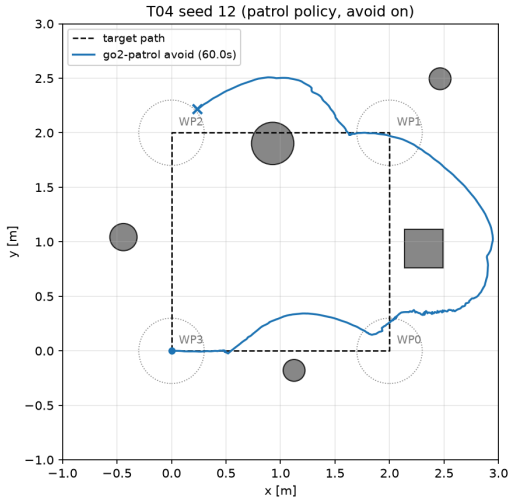


図 7 左: v1、seed 12。経路上の大きな円柱の前で振動し、大回りして WP2 手前で時間切れ。

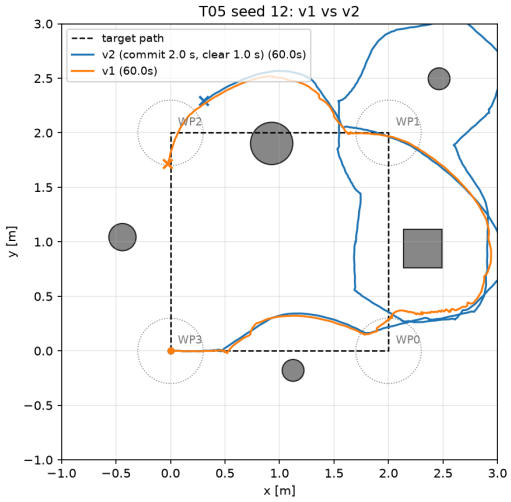


図 7 右: v2、同じ配置。コミット旋回で脇を抜けて 1 周完了。

v2 は 15 配置では 15/15 だったが、45 配置では新たに 2 配置で失敗し、平均所要時間は v1 と変わらず(31.6 → 31.7 s)、**統計的な差は示せなかった**($p = 0.69$)。一方で経路効率は 1.00 → 0.81 に下がった。経路を塞ぐ障害物の数で層別すると構造がはっきりする。

経路上の障害物数	配置数	v1 成功率	v2 成功率	v1 所要 [s]	v2 所要 [s]
0〜1	24	100%	96%	26.6	28.4
2 以上	21	81%	95%	42.7	38.2

所要は失敗 = 60 s を含む平均。v1 で失敗し v2 で成功: 4 配置(いずれも経路上に障害物 2 個以上)。v1 で成功し v2 で失敗: 2 配置(経路脇 0.2〜0.5 m に障害物が 2 個ずつ並び、v2 がそれらにもコミットして外側へ出て到達円に入り損ねる)。

v2 は「難しい配置を落とさない」代わりに「易しい配置で遠回りする」。v1 の振動も v2 の遠回りも、反応的回避が**障害物の向こう側を知らない**ことの裏返しである。そこで v3 では、LiDAR スキャンから 2D 占有格子地図を逐次構築し(log-odds 更新: ヒット点 +0.85、レイ上 −0.4、未知 = 通行可)、Go2 の半径 0.35 m で膨張した上で A* により次のウェイポイントまでの経路を 1 s ごとに再計画し、経路上 0.8 m 先をローカル目標として経路追従に渡す構成にした(実機の costmap / global planner に相当)。v1 の反応的回避は安全停止として下層に残す。地図は既知配置での単体テスト(占有セルが箱の正面 1.28〜1.33 m に出る、期待 1.30 m)で検証した。計算コストは地図更新 0.6 ms/step、A* 4 ms/回で、評価の実行時間は v1/v2 と変わらない。

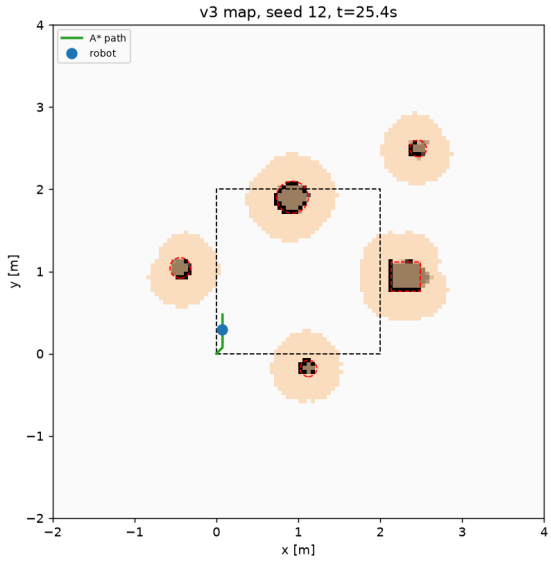


図 8 左: v3 が巡回中に構築した占有格子(seed 12、25.4 s 時点)。黒 = 占有、薄橙 = 膨張領域、赤破線 = 真の障害物。4 方向から見るため輪郭全体が占有になり、偽の占有はない。

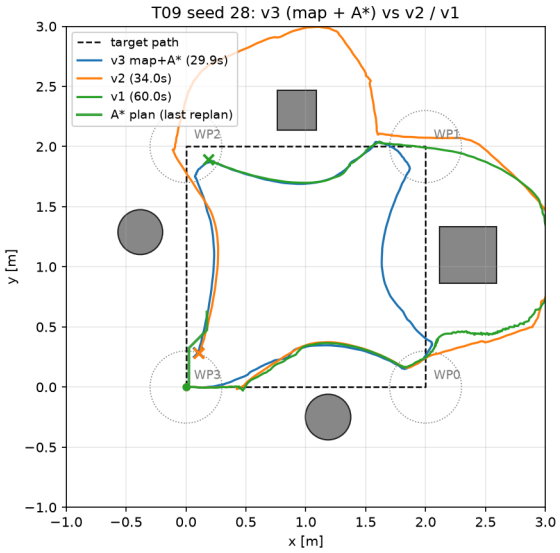


図 8 右: seed 28(経路上に障害物 4 個)。v1(緑)は振動して時間切れ、v2(橙)は外側へ大回り、v3(青)は計画経路に沿って障害物の内側を通り 29.9 s で完走。

経路上の障害物数	配置数	v1 所要 [s]	v2 所要 [s]	v3 所要 [s]	v3 成功
0	7	20.5	24.5	**19.1**	7/7
1	17	29.1	30.0	**21.5**	17/17
2	16	41.5	37.1	**27.4**	16/16
3〜4	5	46.7	41.6	**30.1**	5/5

所要は失敗 = 60 s を含む平均。v1/v2 の失敗配置(seed 12, 25, 26, 28)は v3 で各 2 回ずつ再実行し 8/8 成功・接触 0。45 配置を通して A* の「経路なし」は 0 回で、膨張半径を縮める退化経路は使われなかった。

v3 は全層で最速になり、v1/v2 のトレードオフは解消した。障害物が増えても所要時間の伸びが緩い(0 個 19 s → 3〜4 個 30 s)。v3 が v1 より 10 s 以上遅かった配置が 3 つあるが(いずれも成功)、下層の反応的回避が発動して減速した回と考えられる。改良の各段階は「失敗したエピソードのログを読んで原因を特定し、その原因に対応する層を 1 つ足す」手順で進めた: 振動 → コミット区間(時間の記憶)、遠回り → 地図(空間の記憶)+計画。

9. 工夫した点

1. **層構造の設計**: 学習するのは歩行だけ。ナビゲーションは決定的で説明可能なため、失敗の原因をログから特定して層を直せる(§5, §8 の 3 回の改良はいずれもログ診断から導いた)。v3 で実機 Go2 と同じ「自

己位置 → 地図・計画 → 速度指令 → 歩行」の 3 層が揃い、v1/v2 の挙動は変えずに `--avoid-version` で切り替えられる。

2. **指令範囲の拡張**: 例題の「前進のみ」の限界を 1 配置で実証し、再学習の必要性を 20 配置で確認。
3. **地面除去と単体検証**: センサを幾何期待値・Metal/CPU 一致・歩行中の偽ヒット 0 本で検証してから制御に使った。
4. **評価設計**: 衝突判定の欠陥(LiDAR では正面押し付けを検出できない)を実測で見つけて主指標を変更。対応あり検定と信頼区間、非決定性の切り分け。「期待と違う結果」(既定方策 + 回避則で接触率が上がる、v2 が平均では改善しない、v3 の成功率の差は検定で示せない)をそのまま報告。
5. **計算資源**: Mac 1 台で完結。学習は Metal(4096 環境で 7 万 steps/s、300 it = 9 分)、1 環境の評価は CPU(Metal より 10 倍速いことを実測して切替)。180 エピソードの評価(4 条件 × 45 配置)も同じ環境で完結。
6. **可視化**: ROS なしで MCAP を書き出し、Foxglove で点群・軌跡・指令を実機と同じ形で確認できるようにした。
7. **開発プロセス**: タスク仕様(受け入れ条件つき)→ 実装・計測 → 報告 → レビュー、の反復を 10 回。各報告はリポジトリの `reports/` に残し、設計変更の経緯を追跡できる。

10. 限界と今後

- **真の姿勢を使用**: 自己位置推定(SLAM)は実装していない。v3 の地図更新・A* の始点・ローカル目標の追従はすべてシムの真の姿勢を使うため、本結果は「自己位置が正確なら planner 層はこれで足りる」という上限である。実機では LiDAR SLAM / オドメトリの誤差が地図に累積するので、その影響は未評価。
- **地図と計画の簡略化**: 静止障害物前提の log-odds(動く障害物は消えるまで時間がかかる)、未知セル = 通行可、A* は経路長のみ最小化(膨張領域の外なら障害物のすぐ脇を通る。実機 costmap のような距離に応じたコストは未導入)。障害物の間隔が 1.2 m 以上という生成条件のため「経路なし → 膨張縮小 → 直線」の退化経路は未検証。
- **静止障害物・平坦地形のみ**。動く障害物、段差、センサノイズ、ドメインランダム化は未対応。
- **シミュレータの非決定性**: CPU バックエンドの並列計算に由来すると考えられるが発生源は未特定。評価は配置数で平均化して対処した。
- **2D LiDAR への簡略化**: 実機の 3D LiDAR は低い障害物や段差も見える。本実装では高さ 0.6 m 以上の障害物に限定した。

11. 再現と使用 OSS

環境: macOS(Apple Silicon)、Python 3.11、genesis-world 1.4.4、rsl-rl-lib 5.5、torch 2.14、foxglove-sdk 0.30。`pip install -r requirements.txt` の後、`python go2_train_patrol.py -e go2-patrol -b 4096 --max-iterations 300`(学習)、`python go2_patrol.py -e go2-patrol --ckpt 299 --seed 10 --avoid --avoid-version v3 --backend cpu --out out.mp4 --mcap out.mcap`(巡回・録画。`--avoid-version v1|v2|v3`)、`python eval/run_eval.py ... && python eval/summarize.py`(評価)。詳細はリポジトリの README と `reports/01〜10`。使用 OSS: Genesis(Apache-2.0、Go2 例題 `go2_env.py` / `go2_train.py` を改変・複製し NOTICE に明記)、rsl_rl(BSD-3)、PyTorch、Foxglove SDK(MIT)。本リポジトリは Apache-2.0。